

Practical Week-8

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // malloc allocation
    int *a = (int *)malloc(5 * sizeof(int));
    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }
    for (i = 0; i < 5; i++) {
        a[i] = (i + 1) * 10;
    }
    printf("malloc: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    // calloc allocation
    int *b = (int *)calloc(5, sizeof(int));
    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }
    printf("calloc: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", b[i]);
    }
    printf("\n");

    // realloc allocation
    a = (int *)realloc(a, 10 * sizeof(int));
    if (a == NULL) {
        printf("realloc failed\n");
        free(b);
        return 1;
    }
    for (i = 5; i < 10; i++) {
        a[i] = (i + 1) * 10;
    }
    printf("After realloc: ");
    for (i = 0; i < 10; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    // Free memory
    free(a);
    free(b);
    printf("Memory freed successfully.\n");

    return 0;
}
```

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 memory_leak.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(100 * sizeof(int));
    if (ptr == NULL) {
        return 1;
    }
    printf("Memory allocated successfully.\n");
    // free(ptr); // Intentionally omitted to cause a memory leak

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
imvarls@LAPTOP-VQ4D70L7:~$ nano dynamic_memory.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc dynamic_memory.c -o dynamic_memory
imvarls@LAPTOP-VQ4D70L7:~$ ./dynamic_memory
malloc: 10 20 30 40 50
calloc: 0 0 0 0 0
After realloc: 10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
imvarls@LAPTOP-VQ4D70L7:~$ |

imvarls@LAPTOP-VQ4D70L7: × + ▾
imvarls@LAPTOP-VQ4D70L7:~$ nano memory_leak.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc memory_leak.c -o memory_leak
imvarls@LAPTOP-VQ4D70L7:~$ ./memory_leak
Memory allocated successfully.
```

2. Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int stack_var = 200;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 300;

    printf("Process PID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    printf("Heap address  : %p\n", (void *)&heap_var);

    printf("\nProcess is running...\n");
    printf("Use another terminal to inspect /proc/%d/maps\n", getpid());

    while (1) {
        sleep(1);
    }

    free(heap_var);
    return 0;
}

imvarls@LAPTOP-VQ4D70L7:  ×  +  ▾

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int stack_var = 200;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 300;

    printf("Process PID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    printf("Heap address  : %p\n", (void *)&heap_var);

    printf("\nProcess is running...\n");
    printf("Use another terminal to inspect /proc/%d/maps\n", getpid());

    while (1) {
        sleep(1);
    }

    free(heap_var);
    return 0;
}
```

Output:

imvarls@LAPTOP-VQ4D70L7: × + ▾

```
imvarls@LAPTOP-VQ4D70L7:~$ nano write.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc write.c -o write
imvarls@LAPTOP-VQ4D70L7:~$ ./write
Parent PID: 2630
Data initialized.
Forking process...

Parent PID: 2630
Child PID: 2631
Parent data remains: 1

Child PID: 2631
Child reads data: 0
Child modifying data...
Child modified data: -2
imvarls@LAPTOP-VQ4D70L7:~$ |
```